



**COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS**

CSEB5223: Software Construction and Methods

**Assignment Part 1 (4%)
SEMESTER 1, ACADEMIC YEAR 2026/2027**

SUBJECT CODE : CSEB5223

SUBJECT : Software Construction and Methods

DATE : 23 SEPTEMBER 2026

GROUP NUMBER : Group 1

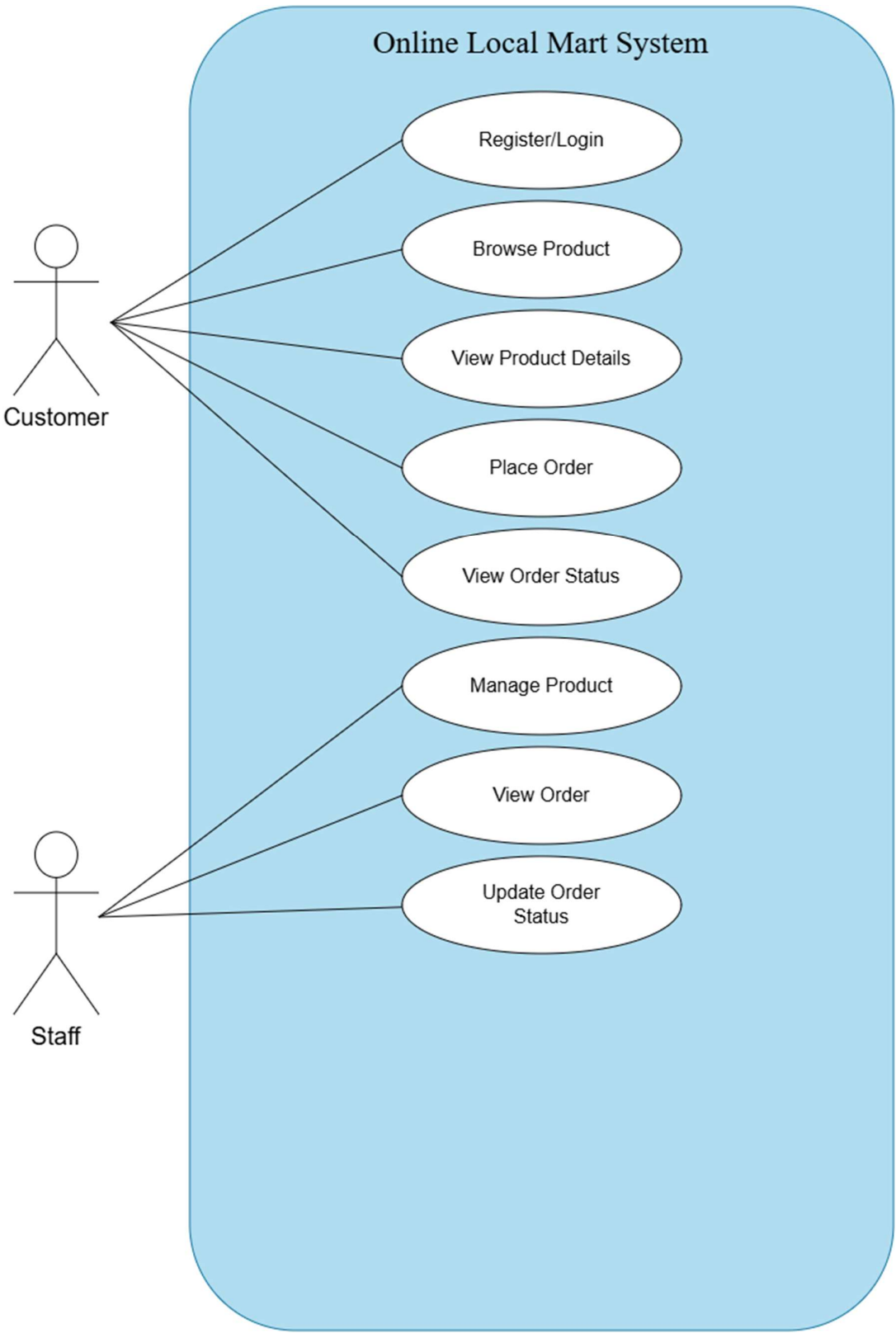
SECTION : 1B

LECTURER : TS. DR. LOO YIM LING

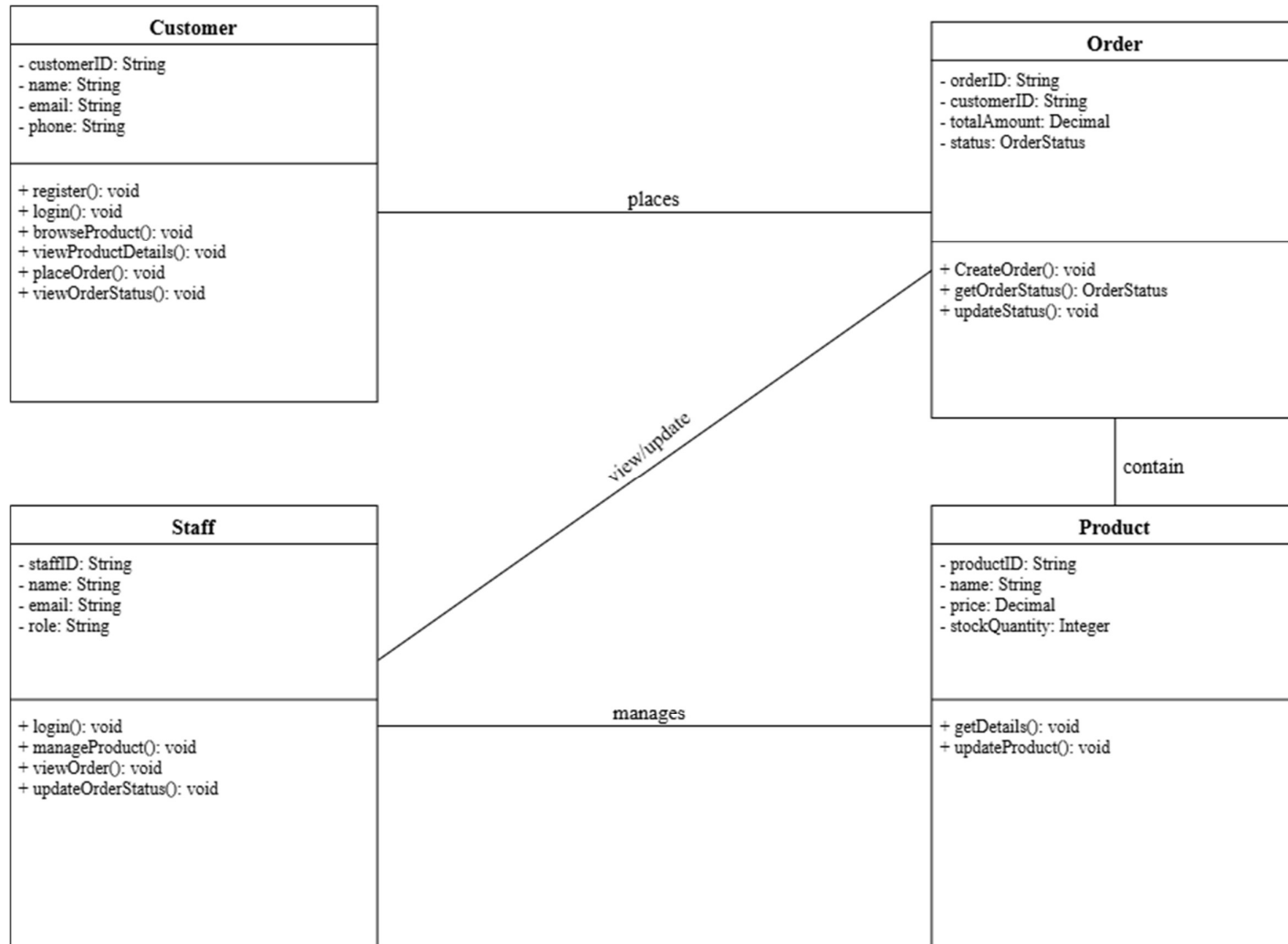
NAME OF MEMBERS :

No.	Full Name	Student ID	Task
1	CHAN WEI HONG	BSW01084678	Activity 1, 2, Report Compile
2	HUSSEIN MUSA MOHAMMED OSMAN	BSW01085603	Activity 3(a)
3	KRISHNACHARANI A/P V KARTHIGEYAN	BSW01084688	Activity 4
4	SYED FARDEYN BIN SALAHUDEEN	SW01082902	Activity 3(b)

Activity 1: Use Case Diagram



UML Class Diagram



Activity 3(a) Naming Conventions – Dictionary

1. General Architectural Casing Conventions

Element	Notation Style	Standard / Syntax Rule	OLMS Implementation Example
Classes / Entities	PascalCase	Nouns representing primary domain entities matching the UML class definitions.	Customer, Staff, Product, Order
Methods / Functions	camelCase	Verbs or verb-object phrases defining operations and behaviors.	browseProduct(), viewOrderStatus(), updateProduct()
Attributes / Variables	camelCase	Descriptive nouns or compound identifiers specifying state.	customerID, stockQuantity, totalAmount
Constants / Enums	UPPER_SNAKE_CASE	Immutable state definitions and enumerations.	OrderStatus.PENDING, OrderStatus.DELIVERED
Database Tables	snake_case (Plural)	Plural nouns representing persistent relational tables.	customers, staffs, products, orders
Database Columns	snake_case (Singular)	Lowercase identifiers; primary and foreign keys suffixed with _id.	customer_id, product_id, stock_quantity
Source Code Files	PascalCase	Filenames must strictly mirror the class name they encapsulate.	Customer.php / Customer.js, Product.php

2. Domain Entities & Attributes Dictionary

Directly derived from the OLMS UML specification to ensure uniform variable naming across UI forms, backend logic, and database schemas:

Entity	Attribute Name (Code)	Database Column	Data Type	Description
Customer	customer ID	customer_id	String / VARCHAR (50)	Unique primary identifier for a customer account.
	name	name	String / VARCHAR (100)	Full legal name of the customer.
	email	email	String / VARCHAR (100)	Registered customer email address (unique).
	phone	phone	String / VARCHAR (20)	Contact phone number for delivery coordination.
Staff	staffID	staff_id	String / VARCHAR (50)	Unique primary identifier for staff/mart admin.
	name	name	String / VARCHAR (100)	Full name of the staff member.
	email	email	String / VARCHAR (100)	Official mart personnel email address.
	role	role	String / VARCHAR (50)	Access role assigned (e.g., Manager, Clerk).

Entity	Attribute Name (Code)	Database Column	Data Type	Description
Product	productID	product_id	String / VARCHAR (50)	Unique inventory SKU / product identifier.
	name	name	String / VARCHAR (150)	Commercial name of the listed product.
	price	price	Decimal (10,2)	Unit selling price of the product.
	stockQuantity	stock_quantity	Integer / INT	Current units available in local mart inventory.
Order	orderID	order_id	String / VARCHAR (50)	Unique invoice / order tracking code.
	customerID	customer_id	String / VARCHAR (50)	Foreign key referencing Customer.customer ID.
	totalAmount	total_amount	Decimal (10,2)	Final calculated financial cost of the order.
	status	status	OrderStatus (Enum / VARCHAR)	Current delivery/processing status of the order.

3. Method & Function Dictionary

Derived directly from the Use Case actions and UML Class operations:

Method Signature	Class Origin	Access	Return Type	Description / Responsibility
register()	Customer	public (+)	void	Creates and validates a new customer account.
login()	Customer, Staff	public (+)	void	Authenticates user credentials and initiates session.
browseProduct()	Customer	public (+)	void	Queries and displays active products to the customer.
viewProductDetails()	Customer	public (+)	void	Displays granular specifications, stock, and price for a single product.
placeOrder()	Customer	public (+)	void	Converts selected items into a pending customer order.
viewOrderStatus()	Customer	public (+)	void	Fetches the current fulfillment state of customer orders.
manageProduct()	Staff	public (+)	void	Coordinates CRUD operations over the product inventory.
viewOrder()	Staff	public (+)	void	Retrieves orders across the mart for processing.

Method Signature	Class Origin	Access	Return Type	Description / Responsibility
updateOrderStatus() ()	Staff, Order	public (+)	void	Mutates order lifecycle (e.g., Pending $\rightarrow$ Shipped $\rightarrow$ Delivered).
getDetails() ()	Product	public (+)	void	Returns internal product properties for viewing.
updateProduct() ()	Product	public (+)	void	Mutates existing product price, name, or stock quantity.
createOrder() ()	Order	public (+)	void	Instantiates and persists a new order record.
getOrderStatus() ()	Order	public (+)	OrderStatus	Queries the current enum value of the order status.

4. Prohibited Terminology & Synonyms Rule

To prevent inconsistencies across team pull requests, only the official terms from the diagram may be used:

- Use Customer, **never** User, Buyer, or Client.
- Use Staff, **never** Admin, Merchant, Employee, or Seller.
- Use Product, **never** Item, Good, or Merchandise.
- Use stockQuantity, **never** qty, count, stock, or amount.
- Use totalAmount, **never** total, price, cost, or bill.

Activity 3(b): Define Functions and Modularity

Function and Method Modularity

Customer Class

- register() creates a new account for the customer with information such as name, email and phone number.
- login() verifies customer's login details before allowing access to the system.
- browseProduct() Search and display any available products for the customer.
- viewProductDetails() shows detailed information for a selected product, such as product name, price, and stock quantity.
- placeOrder() creates an order after the customer selects products to purchase.
- viewOrderStatus() displays the current status of the customer's order, such as Pending, Processing, Shipped, or Completed.

Staff Class

- login() verifies staff login credentials.
- manageProduct() coordinates product management tasks, such as adding, editing, or removing a product.
- viewOrder() retrieves and displays customer order information for staff.
- updateOrderStatus() changes the status of an order after it has been processed.
- Product Class
- getDetails() returns the details of a product for customers or staff to view.
- updateProduct() updates product information, such as the product name, price, or stock quantity.

Order Class

- CreateOrder() creates a new order when a customer places an order.
- getOrderStatus() returns the current order status for the customer or staff.
- updateStatus() changes the order status when staff process the order.

Activity 4: Define Communication and Continuous Integration Rules

a. Communication Rules

Category	Policy / Standard	Operational Details
Communication Channels	Centralized Platform	All project-related communication should be conducted through the agreed team communication platform.
Task Management	Issue Tracking via GitHub	GitHub Issues should be used to report bugs, assign development tasks, and track project-related problems.
Defect Reporting	Structured Issue Descriptions	Team members should clearly describe the issue, expected outcome, and relevant module when creating an issue.
Progress Reporting	Transparency & Updates	Team members should provide regular updates on their assigned tasks and inform the team early about any problems or delays.
System Decisions	Prior Technical Consensus	Technical decisions that affect the system architecture, database, coding standards, or development workflow should be discussed with the team before implementation.
Pull Request Content	Clear Context & Traceability	Pull Requests should include a clear description of the changes made and the related issue or task where applicable.
Code Review	Mandatory Peer Review	Team members should review and respond to Pull Request comments before the code is merged.
Knowledge Base	GitHub Documentation	Important project decisions and changes should be documented in GitHub so that they can be referred to by all team members.
Domain Standards	Dictionary Alignment	Team members should use the approved domain terminology consistently, such as Customer, Staff, Product, Order, stockQuantity, and totalAmount.

Category	Policy / Standard	Operational Details
Workflow Governance	Standard Alterations	Any changes to the agreed coding standards, system structure, or development workflow should be communicated to the entire team.

b. Continuous Integration Rules

Pipeline Stage	CI Standard	Enforcement Criteria
Branching Strategy	Isolated Feature Branches	All new features and fixes must be developed using a separate <code>feature</code> branch.
Branch Protection	Protected Primary Branches	Developers must not directly push changes to the <code>main</code> or <code>dev</code> branches.
Staging Workflow	Targeted Pull Requests	Completed features must be submitted through a Pull Request to the <code>dev</code> branch.
Review Gate	Minimum Approvals	Each Pull Request must be reviewed by at least one other team member before merging.
Automation Trigger	Automated CI Runs	CI checks must run automatically when code is pushed to a feature branch or when a Pull Request is created or updated.
Build & Test	Integrity Verification	CI must verify that the project can be built successfully and that automated tests, where available, pass successfully.
Quality Gate	Zero-Failure Policy	Code that fails the required CI checks must not be merged until the errors are fixed.
Integration	Staging Verification	The <code>dev</code> branch is used to integrate, and test completed features before they are released to <code>main</code> .
Release Management	Stable Main Branch	The <code>main</code> branch must contain only stable and production-ready code.
Regression Prevention	Core Module Safeguards	Changes merged into <code>dev</code> should not break existing system modules such as Customer, Staff, Product, and Order functions.
Coding Standards	Pre-PR Compliance	Developers should ensure that their code follows the approved naming conventions and modularity rules before creating a Pull Request.
Code Modularity	Function Length Limit	Functions should remain within the agreed maximum of 30 lines unless there is a documented reason for exceeding the limit.

Pipeline Stage	CI Standard	Enforcement Criteria
Defect Ownership	Responsible Triage	Any failed CI build or test should be investigated and resolved by the developer responsible for the change.